

TLaser User Manual

Welcome to TLaser, the Digital Twin Control Center for telecom diode lasers. This manual provides installation instructions, mathematical formulations, execution steps for the simulation and training pipelines, and details on online parameter calibration.

Quick start

From PowerShell in the upgraded TLaser folder, with dependencies installed:

```
./run_tlaser.ps1
```

Open <http://localhost:8501> manually. For port 8502, use `./run_tlaser.ps1 -Port 8502`. Keep the terminal open; Ctrl+C stops the server. See section 7 for configuration, save/load, and troubleshooting.

1. Project Overview

Updated 2026-10-09. TLaser is a reduced-model prototype for a fixed facet-mirror ridge edge emitter at a steady-state CW operating point. CW is a preset, not a separate architecture. It combines:

1. Quasi-3D Simulator Core: Solves longitudinal carrier density and wave propagation.
2. Physics-Informed Neural Network (PINN): Maps seven design/operating inputs to 105 outputs. The current synthetic-data model has no held-out accuracy evaluation or verified sub-5-ms end-to-end timing guarantee.
3. Monitored L-I-V Parameter Calibration: Fits internal parameter drift against real-time measured Light-Current-Voltage curves.

1.1 Engineering Mapping Storyline

To clearly articulate TLaser's position in the industrial telecom photonics value chain, the platform maps a complete engineering path from products down to digital twin parameters:

1. Optical Module Level: Standard SFP/QSFP transceivers plugged into network switches. System testing provides measured L-I-V (Light-Current-Voltage) curves.
2. Assembly Internal Subsystems: Subsystems like the Transmitter Optical Sub-Assembly (TOSA), laser driver, and thermoelectric coolers (TEC) dictate operational states.
3. Laser Chip Level: The bare semiconductor laser chip inside the TOSA. Waveguide dimensions (L, w, d) define its boundaries.
4. Forward model: Uses reduced longitudinal carrier/optical equations, temperature-dependent constants, and electrical parasitics. The edge model has no self-heating feedback or executable FEM/TCAD coupling.
5. Inverse model: Fits selected α_i , Γ , C_{mult} , R_{series} , and R_{shunt} values while holding geometry, facets, and temperature fixed. It does not estimate dimensions or edge thermal resistance.

2. Environment Setup

Configure a local Python virtual environment to manage dependencies safely:

```
# Clone the repository
git clone https://github.com/ZhenwenWan/TLaser.git
cd TLaser

# Create virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1 # On Windows PowerShell

# Install required libraries
pip install -r requirements.txt
```

Troubleshooting Environment Setup

* PyTorch CPU Wheel issues: On systems without a CUDA GPU, pip might attempt to install CUDA-enabled PyTorch which can be large or fail. Use:

```
pip install torch --extra-index-url https://download.pytorch.org/whl/cpu
```

* Missing matplotlib or OpenCV: Verify that your path points to the virtual environment interpreter (`.venv\Scripts\python.exe`) rather than the global system Python.

3. Synthetic Dataset Generation

The dataset generator runs random parameter sweeps over the 7D design domain.

Simulator Nature: The simulator is a reduced synthetic longitudinal model. Sampling bounds are not evidence of measured-device accuracy. Active width is not established as etched ridge width; effective thickness is not an epitaxial stack.

Sweep Boundaries

- * Mirror reflectivities: R_1 in $[0.1, 0.95]$, R_2 in $[0.05, 0.5]$
- * Cavity dimensions: Length L in $[100, 1000]$ microm, Ridge Width w in $[1.5, 4.0]$ microm, Thickness d in $[0.1, 0.5]$ microm
- * Environmental inputs: Temperature T_0 in $[250, 360]$ K, Injection Current I_{active} in $[0.01, 0.5]$ A

Execution Commands

* Run full sweeps (1500 samples):

```
python simulator/generate_dataset.py --num-samples 1500
```

* Run quick smoke test:

```
python simulator/generate_dataset.py --smoke-test --output-dir data/smoke_test
```

Outputs are saved in `data/` as `pinn_inputs.npy` and `pinn_targets.npy`, alongside `pinn_dataset_metadata.json`.

4. Physics-Informed Neural Network Training

The PINN model trains a surrogate to reproduce the laser behavior under physical boundary penalties.

Optimization Penalties

1. Data Loss: Matches predicted output power, WPE, current, and profile points against simulator targets.

2. Carrier Rate Equation Residual:

$$G_{\{inj\}} - R_{\{rec\}}(N(z)) - R_{\{stim\}}(N(z), P(z)) = 0$$

3. Photon Propagation Wave Residual (Second-Order Approximation):

$$d^2P/dz^2 - (\Gamma g(z) - \alpha_i)^2 P(z) = 0$$

Note: This prototype uses a reduced total-power residual instead of predicting forward and backward field components separately. It does not establish full-field accuracy.

4. Laplacian Smoothness Regularization: Smooths predicted carrier and power curves.

Execution Commands

* Run full training (600 epochs):

```
python surrogate/train.py --epochs 600
```

* Run quick smoke test:

```
python surrogate/train.py --smoke-test --output-dir data/smoke_test
```

Saves weights to `data/pinn_laser_model.pt` and training convergence history to `data/pinn_training_loss.svg`.

5. Parameter Calibration Loop

The shared device workflow uses the selected configuration for fitting. Select Parameter Calibration Loop, then explicitly choose Synthetic demo or upload JSON. The demo changes only selected parameters and is not measured-device validation.

Input Data Schemas

* JSON File: Requires known active-region current in amperes, not ordinary terminal current.

Arrays need 5–200 finite points, equal lengths, at least five distinct currents within 0.01–0.5 A, positive voltage, and nonnegative informative power. Metadata must match the selected device. This example shows the format only; replace values with measurements:

```
{
  "current_semantics": "active_region",
  "current_A": [0.05, 0.10, 0.20, 0.30, 0.40],
  "voltage_V": [1.5, 1.6, 1.7, 1.8, 1.9],
  "optical_power_W": [0.001, 0.002, 0.003, 0.004, 0.005],
  "metadata": {
    "L_um": 300.0, "w_um": 2.8, "d_um": 0.342,
    "R1": 0.90, "R2": 0.05, "T0": 298.0
  }
}
```

* The new dashboard accepts JSON. CSV handling belongs to the legacy command-line path, which silently supplies default geometry and is not the shared workflow.

Fit, compare, and apply

Start with a small subset; series resistance is selected by default. Select Run calibration, then inspect original/estimated parameters, LIV curves, optimizer status, before/after normalized loss, bounds, and local sensitivity. Geometry outlines remain identical because dimensions are fixed.

A fit can be applied only when optimization succeeds, loss improves below 0.01, no bound or insensitive-parameter flag occurs, and local Jacobian condition is below 1,000,000. These diagnostics do not prove global identifiability. Failed candidates can be exported for review but cannot be applied through the fit button.

Select Use fitted constants for simulator prediction, return to predictions, and choose Physical simulator. The seven-input Nominal PINN remains nominal and ignores fitted constants. Download the estimated configuration and complete calibration report to keep original/estimated snapshots and measurement provenance. Current session results do not overwrite historical shared calibration files.

Legacy command-line commands

These older commands write shared results under `data/` and do not use the new session configuration.

* Run calibration on mock monitoring data:

```
python calibration/calibrate.py
```

* Run calibration on external monitored file:

```
python calibration/calibrate.py --data-file data/monitored_liv.json
```

Saves fitted constants to `data/calibrated_params.json` and generates the L-I-V fit comparison chart in `data/calibration_fit.svg`.

6. Automated Pipeline Verification

To validate the shared device workflow without overwriting model artifacts, run:

```
python -m unittest discover -s tests -p test_device_workflow.py -v
```

The seven tests cover configuration round trips, invalid-data rejection, dimension/backend agreement, known resistance recovery, fixed geometry, failed-fit rejection, workflow/language state, and demo-to-applied prediction.

The legacy pipeline below generates files and updates a verification report; it is not a read-only accuracy audit:

```
python verify_pipeline.py
```

7. Interactive App Dashboard

Open PowerShell in the upgraded TLaser project folder, with dependencies installed, and run:

```
./run_tlaser.ps1
```

Open <http://localhost:8501> manually. The launcher prints the address; it does not open a browser. Keep the terminal running; Ctrl+C stops the service. If the port is occupied:

```
./run_tlaser.ps1 -Port 8502
```

Then open <http://localhost:8502>. The development preview uses 8502; the launcher default is 8501. Alternatively, from an activated working environment:

```
python -m streamlit run app.py
```

The launcher can reuse installed packages with Codex's bundled Python if this checkout's old Windows Store interpreter is unavailable. Other computers need a working virtual environment. If script execution is blocked, follow local execution-policy rules or use the alternative Python command.

Configure, save, and restore

Select EN or CN and Edge-emitting Diode Laser in the sidebar; expand the sidebar on a narrow screen. Enter L, effective w/d, R1/R2, temperature, and active-region current, then select Update device to commit edits. The labeled top view and cross-section update together. Substrate/cladding are fixed illustrative context; thickness is exaggerated and display scales differ by axis.

Expand Save / load device configuration to download versioned JSON. Restore by uploading a configuration and selecting Load configuration. Invalid versions, architectures, nonfinite values, and out-of-range inputs are rejected. Configuration survives workflow/language switches; updating or loading a design clears the previous fit and applied flag. Downloaded files are persistent; browser-session state is not a permanent database.

In predictions, choose Nominal PINN or Physical simulator. Review both-facet total optical power, WPE, total terminal current, and 51-point profiles. Active-region current excludes leakage; terminal current includes the model's leakage contribution.

Troubleshooting and limits

- Missing dependencies: activate a working environment and install `requirements.txt`; `jsonschema` is required for legacy VCSEL import.
- Missing nominal weights: `geometry` and `simulator` remain available. Nominal prediction needs `data/pinn_laser_model.pt` and `data/pinn_scale_params.npz`.
- Metadata mismatch: match the selected device to the measurement; do not relabel measurements merely to force a fit.
- Background: restart the server to read `.streamlit/config.toml`. The original navy palette is retained across the dashboard, schematic, plots, and PDF guides.
- Dataset generation/training without a separate output directory overwrites production data/artifacts. Smoke commands above use `data/smoke_test` instead.
- DFB grating physics, EML absorber/modulation, inverse geometry estimation, measured-device validation, reliable solver convergence certificates, and held-out surrogate evaluation remain future work. VCSEL is a separate legacy reduced-model demonstration; its G multiplier is not identifiable from current LIV outputs.
- The public project webpage is a presentation/documentation surface. Computation runs locally; public computation hosting, authentication, and multiuser storage are not implemented.